

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

Отчет
по лабораторной работе № 2
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Оценка времени выполнения программ»

Выполнил
студент группы 24ВВВ2:

Воеводин И.А.

Гуляевский Д.С.

Каташов К.А.

Приняли

Юрова О.В.

Деев М.В.

Пенза, 2025

Цель работы

Провести исследование эффективности различных алгоритмов и операций (перемножения матриц и сортировки массивов) путем оценки их временной сложности с использованием теоретических моделей (О-символика), измерения реального времени выполнения на различных входных данных (матрицы и массивы разного размера и структуры), а также визуализации результатов для сопоставления теории с практикой

Лабораторное задание

Задание 1:

- Вычислить порядок сложности программы (О-символику).
- Оценить время выполнения программы и кода, выполняющего перемножение
 - матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000
 - Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Задание 2:

- Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.
- Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.
- Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.
- Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.
- Оценить время работы стандартной функции `qsort`, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

Описание метода решения

Для оценки временной сложности алгоритма анализируют количество операций в зависимости от размера входных данных N . Подсчитывают шаги во вложенных циклах, выделяют главный член с наибольшим ростом (например, N^3 для тройного цикла) и обозначают сложность как $O(N^3)$, игнорируя константы и менее значимые слагаемые.

Потом измеряют реальное время выполнения программы с помощью функций из `time.h` для разных размеров данных и сравнивают экспериментальные результаты с теоретической оценкой.

Выделение памяти - операции `malloc` выполняются константное количество раз (3 раза), временная сложность — $O(1)$.

Псевдокод

1-ое задание
алг умножение_матриц

дано
размер матрицы N (вводится пользователем)
две матрицы A и B размера $N \times N$
результат умножения в матрицу C того же размера

надо
- сгенерировать две случайные матрицы A и B
- вычислить произведение матриц $C = A \times B$
- измерить время выполнения операций

нач цел
матрицы A , B , C размером $N \times N$
переменные i , j , k , время

вывести "Размер матрицы"
ввести N

инициализировать генератор случайных чисел

нц для i от 0 до $N-1$
нц для j от 0 до $N-1$
 $A[i, j] :=$ случайное число от 0 до 100
 $B[i, j] :=$ случайное число от 0 до 100
кц
кц

записать время начала вычислений

нц для i от 0 до N-1

нц для j от 0 до N-1

$C[i, j] := 0$

нц для k от 0 до N-1

$C[i, j] := C[i, j] + A[i, k] * B[k, j]$

кц

кц

кц

записать время окончания вычислений

вычислить время выполнения как разницу времени конца и начала

вывести "Размер матриц: " + N + "x" + N

вывести "Время умножения: " + время выполнения + " секунд"

кОН

2-ое задание
алг сравнение_сортировок

дано
размер массива SIZE = 15206
массивы originalArray, shellArray, quickArray размером SIZE
процедуры сортировок: Shell и Быстрая

надо измерить и вывести время работы каждой сортировки на массиве

нач цел
массивы originalArray, shellArray, quickArray размером SIZE
время start_Sort_Shell, end_Sort_Shell, start_Sort_Quick, end_Sort_Quick
переменные i, j, gap, temp, pivot, low, high, pi

инициализировать генератор случайных чисел

// Заполнение массива
вывести "Выберите тип массива (1-возрастающий, 2-убывающий, 3-
угловатый)"
ввести choice

если choice = 1 тогда
нц для i от 0 до SIZE-1
 originalArray[i] := i
кц
иначе если choice = 2 тогда
нц для i от 0 до SIZE-1
 originalArray[i] := SIZE - i
кц
иначе если choice = 3 тогда
j := (SIZE-1)/2
нц для i от 0 до j-1
 originalArray[i] := i
кц
j := j + 1
нц для i от j-1 до SIZE-1
 originalArray[i] := j
j := j - 1
кц
все

вывести "Исходный массив (100 элементов):"

// Сортировка Шелла
копировать(originalArray, shellArray, SIZE)

записать время начала в start_Sort_Shell

нц для gap от SIZE/2 до 1 с шагом gap/2

нц для i от gap до SIZE-1

temp := shellArray[i]

j := i

пока j ≥ gap и shellArray[j - gap] > temp

shellArray[j] := shellArray[j - gap]

j := j - gap

кц

shellArray[j] := temp

кц

кц

записать время окончания в end_Sort_Shell

вычислить Sort_Shell_time = end_Sort_Shell - start_Sort_Shell

вывести "Сортировка Шелла завершена за " + Sort_Shell_time/1000.0 + "
секунд"

// Быстрая сортировка

копировать(originalArray, quickArray, SIZE)

записать время начала в start_Sort_Quick

вызвать quickSort(quickArray, 0, SIZE-1)

записать время окончания в end_Sort_Quick

вычислить Sort_Quick_time = end_Sort_Quick - start_Sort_Quick

вывести "Быстрая сортировка завершена за " + Sort_Quick_time/1000.0 + "
секунд"

кон

// Процедура быстрой сортировки

процедура quickSort(массива arr, low, high)

если low < high тогда

pi := partition(arr, low, high)

вызвать quickSort(arr, low, pi-1)

вызвать quickSort(arr, pi+1, high)

все

конец процедуры

// Процедура разделения для быстрой сортировки

процедура partition(массива arr, low, high)

pivot := arr[high]

i := low - 1

нц для j от low до high-1
если $\text{arr}[j] \leq \text{pivot}$ тогда
 $i := i + 1$
 поменять $\text{arr}[i]$ и $\text{arr}[j]$
все
кц

поменять $\text{arr}[i+1]$ и $\text{arr}[\text{high}]$
вернуть $i+1$
конец процедуры

// Процедура копирования массива
процедура копировать(откуда, куда, размер)
нц для i от 0 до размер-1
 $\text{куда}[i] := \text{откуда}[i]$
кц
кон

Листинг

1-ое задание

```
#include <iostream>
#include <vector>
#include <random>
#include <chrono>

// #include <cstdlib>

using namespace std;
using namespace chrono;

//// Генератор случайных чисел
// random_device rd;
// mt19937 gen(rd());
// uniform_real_distribution<double> dis(0.0, 100.0);

// Функция для генерации случайной матрицы
vector<vector<double>> generateRandomMatrix(int rows, int cols) {
    vector<vector<double>> matrix(rows, vector<double>(cols));
    for (int i = 0; i < rows; i++) {
        /*cout << "\n";*/
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = 0 + rand() % ((100 + 1) - 0);
            /*cout << matrix[i][j]<<" ";*/
        }
    }
    return matrix;
}

// Наивное умножение матриц (O(n³))
vector<vector<double>> naiveMatrixMultiply(const vector<vector<double>>& A,
    const vector<vector<double>>& B, int size) {
    /*int n = A.size();
    int m = A[0].size();
    int p = B[0].size();*/

    vector<vector<double>> C(size, vector<double>(size, 0.0));

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            for (int k = 0; k < size; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    return C;
}

int main() {
    srand(time(NULL));
    setlocale(LC_ALL, "");
    int SIZE;
    cout << "Размер матрицы " << endl;
    cin >> SIZE;
```



```

cout << "Генерация матриц " << SIZE << "x" << SIZE << "..." << endl;

// Замер времени генерации
auto start_gen = high_resolution_clock::now();
auto A = generateRandomMatrix(SIZE, SIZE);
auto B = generateRandomMatrix(SIZE, SIZE);
auto end_gen = high_resolution_clock::now();
auto gen_time = duration_cast<milliseconds>(end_gen - start_gen).count();

cout << "Генерация завершена за " << gen_time / 1000.0 << " секунд" << endl;
cout << "Начинается умножение..." << endl;

auto start_mult = high_resolution_clock::now();
auto C = naiveMatrixMultiply(A, B, SIZE);
auto end_mult = high_resolution_clock::now();
auto mult_time = duration_cast<milliseconds>(end_mult - start_mult).count();

double total_seconds = mult_time / 1000.0;

cout << "\nРЕЗУЛЬТАТЫ:" << endl;
cout << string(40, '=') << endl;
cout << "Размер матриц: " << SIZE << "x" << SIZE << endl;
cout << "Время генерации: " << gen_time / 1000.0 << " секунд" << endl;
cout << "Время умножения: " << total_seconds << " секунд" << endl;
cout << "Общее время: " << (gen_time + mult_time) / 1000.0 << " секунд" << endl;

return 0;
}

```

2-ое задание

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <chrono>
#include <iomanip>
using namespace std;
using namespace chrono;

double getTimeSeconds() {
    return duration_cast<duration<double>>(
        high_resolution_clock::now().time_since_epoch()
    ).count();
}

int compare(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}

void shellSort(int* arr, int n) {
    for (int gap = n / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < n; i++) {
            int temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap) {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
    }
}

int partition(int* arr, int low, int high) {
    int mid = low + (high - low) / 2;
    int pivot = arr[mid];
    swap(arr[mid], arr[high]);

    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(int* arr, int low, int high) {
    while (low < high) {
        int pi = partition(arr, low, high);

        if (pi - low < high - pi) {
            quickSort(arr, low, pi - 1);
            low = pi + 1;
        }
        else {
            quickSort(arr, pi + 1, high);
        }
    }
}
```

```

        high = pi - 1;
    }
}
}

void printArray(const int* arr, int n) {
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
        if ((i + 1) % 10 == 0) cout << endl;
    }
    cout << endl;
}

int* createAndFillArray(int n, int choice) {
    int* arr = (int*)malloc(n * sizeof(int));
    if (arr == nullptr) {
        cerr << "Ошибка выделения памяти!" << endl;
        return nullptr;
    }

    switch (choice) {
        case 1:
            for (int i = 0; i < n; i++) arr[i] = i;
            break;
        case 2:
            for (int i = 0; i < n; i++) arr[i] = n - i;
            break;
        case 3:
        {
            int j = (n - 1) / 2;
            for (int i = 0; i < j; i++) arr[i] = i;
            for (int i = j; i < n; i++) arr[i] = n - i;
        }
            break;
        case 4:
            for (int i = 0; i < n; i++) arr[i] = rand() % 1000;
            break;
    }

    return arr;
}

void copyArray(const int* source, int* dest, int n) {
    for (int i = 0; i < n; i++) dest[i] = source[i];
}

double testSort(void (*sortFunc)(int*, int), const int* arr, int n) {
    int* testArr = (int*)malloc(n * sizeof(int));
    if (testArr == nullptr) {
        cerr << "Ошибка выделения памяти!" << endl;
        return 0;
    }
    copyArray(arr, testArr, n);

    double start = getSeconds();
    sortFunc(testArr, n);
    double end = getSeconds();

    free(testArr);
}

```

```

    return end - start;
}

double testQuickSort(const int* arr, int n) {
    int* testArr = (int*)malloc(n * sizeof(int));
    if (testArr == nullptr) {
        cerr << "Ошибка выделения памяти!" << endl;
        return 0;
    }
    copyArray(arr, testArr, n);

    double start = getTimeSeconds();
    quickSort(testArr, 0, n - 1);
    double end = getTimeSeconds();

    free(testArr);
    return end - start;
}

double testQsort(const int* arr, int n) {
    int* testArr = (int*)malloc(n * sizeof(int));
    if (testArr == nullptr) {
        cerr << "Ошибка выделения памяти!" << endl;
        return 0;
    }
    copyArray(arr, testArr, n);

    double start = getTimeSeconds();
    qsort(testArr, n, sizeof(int), compare);
    double end = getTimeSeconds();

    free(testArr);
    return end - start;
}

void printTableHeader() {
    cout << setw(20) << "Тип массива"
        << setw(20) << "Shell Sort (сек)"
        << setw(20) << "Quick Sort (сек)"
        << setw(20) << "qsort() (сек)"
        << endl;
    cout << string(80, '-') << endl;
}

void printTableRow(const string& arrayType, double shellTime,
    double quickTime, double qsortTime) {
    cout << setw(20) << arrayType
        << setw(20) << fixed << setprecision(6) << shellTime
        << setw(20) << fixed << setprecision(6) << quickTime
        << setw(20) << fixed << setprecision(6) << qsortTime
        << endl;
}

void runTests(int size) {
    cout << "\nРЕЗУЛЬТАТЫ ДЛЯ РАЗМЕРА МАССИВА: " << size << endl;

    string arrayTypes[] = { "Упорядоченный", "Обратный", "Частичный", "Случайный" };

```

```

printTableHeader();

for (int i = 1; i <= 4; i++) {
    int* originalArray = createAndFillArray(size, i);
    if (originalArray == nullptr) {
        cerr << "Ошибка выделения памяти для массива размера " << size << endl;
        continue;
    }

    double shellTime = testSort(shellSort, originalArray, size);
    double quickTime = testQuickSort(originalArray, size);
    double qsortTime = testQsort(originalArray, size);

    printTableRow(arrayTypes[i - 1], shellTime, quickTime, qsortTime);

    free(originalArray);
}

int main() {
    setlocale(LC_ALL, "Russian");
    srand(time(NULL));

    cout << "СРАВНЕНИЕ АЛГОРИТМОВ СОРТИРОВКИ" << endl;
    cout << "===== " << endl;

    runTests(100);
    runTests(1000);
    runTests(500000);

    return 0;
}

```

Результат работы программы 1-ое задание

```
Размер матрицы
100
Генерация матриц 100x100...
Генерация завершена за 0 секунд
Начинается умножение...

РЕЗУЛЬТАТЫ:
=====
Размер матриц: 100x100
Время генерации: 0 секунд
Время умножения: 0.024 секунд
Общее время: 0.024 секунд

C:\Users\user\source\repos\lab2 alg\x64\Debug\lab2 alg.exe (процесс 8296) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```

100

```
Размер матрицы
200
Генерация матриц 200x200...
Генерация завершена за 0.003 секунд
Начинается умножение...

РЕЗУЛЬТАТЫ:
=====
Размер матриц: 200x200
Время генерации: 0.003 секунд
Время умножения: 0.197 секунд
Общее время: 0.2 секунд

C:\Users\user\source\repos\lab2 alg\x64\Debug\lab2 alg.exe (процесс 16240) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно:
```

200

```
Размер матрицы
400
Генерация матриц 400x400...
Генерация завершена за 0.012 секунд
Начинается умножение...

РЕЗУЛЬТАТЫ:
=====
Размер матриц: 400x400
Время генерации: 0.012 секунд
Время умножения: 1.456 секунд
Общее время: 1.468 секунд

C:\Users\user\source\repos\lab2 alg\x64\Debug\lab2 alg.exe (процесс 12296) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно: █
```

400

```
Размер матрицы
1000
Генерация матриц 1000x1000...
Генерация завершена за 0.075 секунд
Начинается умножение...

РЕЗУЛЬТАТЫ:
=====
Размер матриц: 1000x1000
Время генерации: 0.075 секунд
Время умножения: 30.715 секунд
Общее время: 30.79 секунд

C:\Users\user\source\repos\lab2 alg\x64\Debug\lab2 alg.exe (процесс 7848) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно: █
```

1000

```
Размер матрицы
2000
Генерация матриц 2000x2000...
Генерация завершена за 0.293 секунд
Начинается умножение...

РЕЗУЛЬТАТЫ:
=====
Размер матриц: 2000x2000
Время генерации: 0.293 секунд
Время умножения: 290.949 секунд
Общее время: 291.242 секунд

C:\Users\user\source\repos\lab2 alg\x64\Debug\lab2 alg.exe (процесс 2168) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно: █
```

2000

```
Размер матрицы
4000
Генерация матриц 4000x4000...
Генерация завершена за 1.221 секунд
Начинается умножение...

РЕЗУЛЬТАТЫ:
=====
Размер матриц: 4000x4000
Время генерации: 1.221 секунд
Время умножения: 4506 секунд
Общее время: 4507.22 секунд

C:\Users\user\source\repos\lab2 alg\x64\Debug\lab2 alg.exe (процесс 19192) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно: █
```

4000

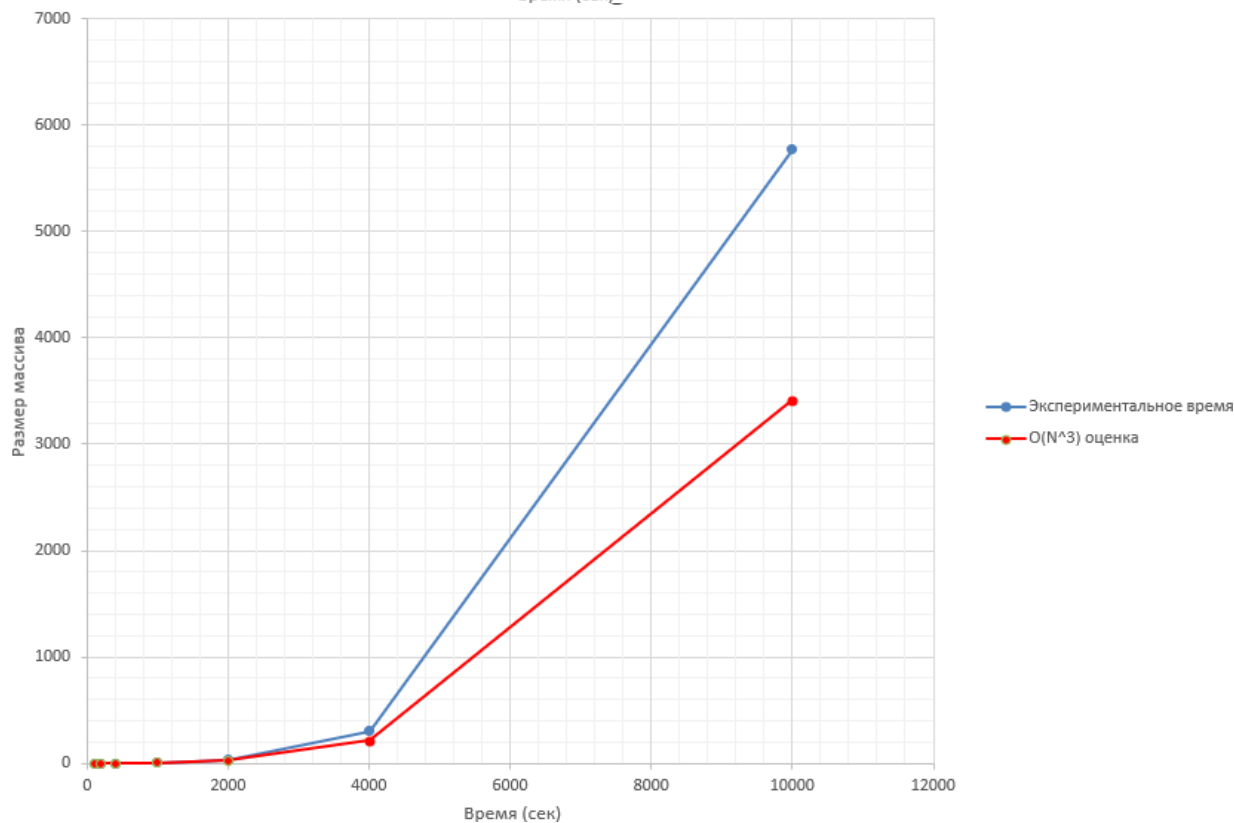
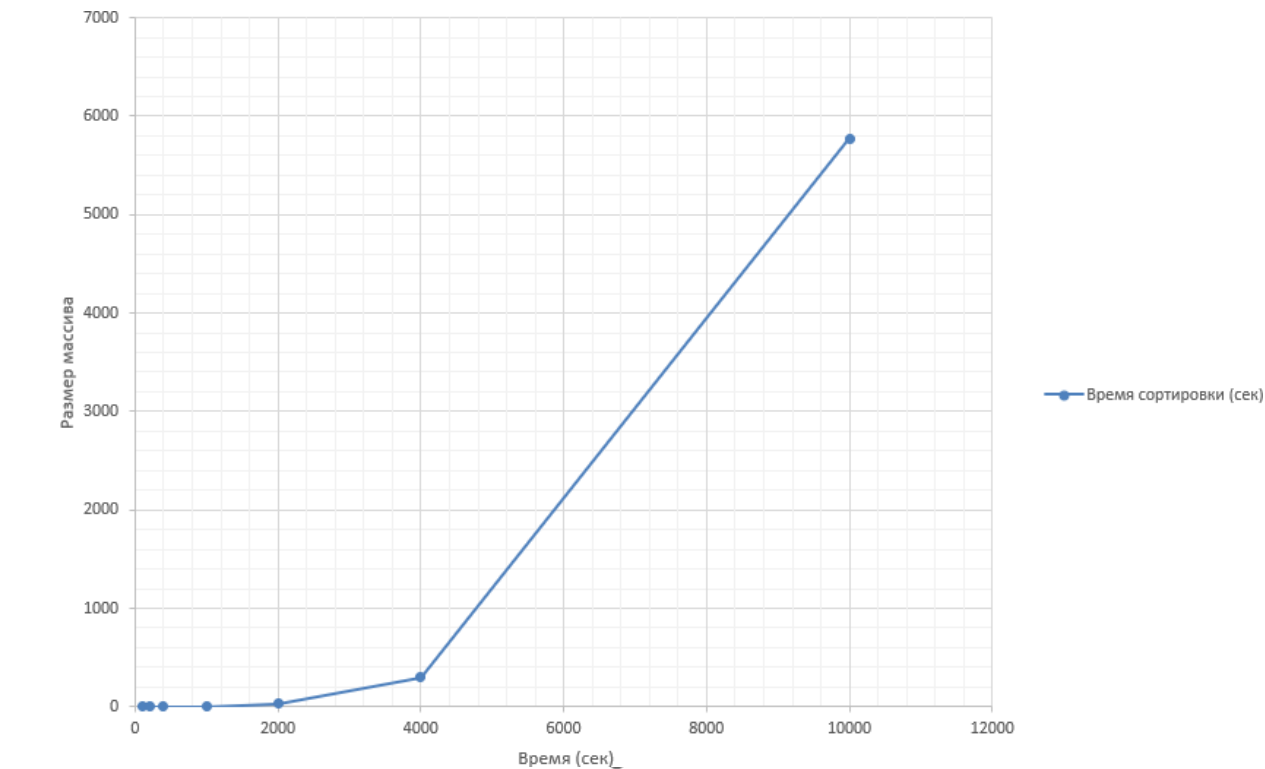
```
Размер матрицы
10000
Генерация матриц 10000x10000...
Генерация завершена за 6.63 секунд
Начинается умножение...

РЕЗУЛЬТАТЫ:
=====
Размер матриц: 10000x10000
Время генерации: 6.63 секунд
Время умножения: 44524.4 секунд
Общее время: 44531 секунд

C:\Users\user\source\repos\lab2 alg\x64\Debug\lab2 alg.exe (процесс 13224) завершил работу с кодом 0 (0x0).
Нажмите любую клавишу, чтобы закрыть это окно: █
```

10000

Размер массива (N)	Время сортировки (сек)
100	0.01
200	0.019
400	0.168
1000	3.416
2000	32.09
4000	300.672
10000	5773.017



2-ое задание

СРАВНЕНИЕ АЛГОРИТМОВ СОРТИРОВКИ

=====

РЕЗУЛЬТАТЫ ДЛЯ РАЗМЕРА МАССИВА: 100

Тип массива	Shell Sort (сек)	Quick Sort (сек)	qsort() (сек)
Упорядоченный	0.000002	0.000003	0.000007
Обратный	0.000003	0.000005	0.000006
Частичный	0.000004	0.000012	0.000016
Случайный	0.000005	0.000007	0.000009

РЕЗУЛЬТАТЫ ДЛЯ РАЗМЕРА МАССИВА: 1000

Тип массива	Shell Sort (сек)	Quick Sort (сек)	qsort() (сек)
Упорядоченный	0.000019	0.000042	0.000046
Обратный	0.000030	0.000052	0.000051
Частичный	0.000026	0.000254	0.000141
Случайный	0.000080	0.000084	0.000109

РЕЗУЛЬТАТЫ ДЛЯ РАЗМЕРА МАССИВА: 500000

Тип массива	Shell Sort (сек)	Quick Sort (сек)	qsort() (сек)
Упорядоченный	0.021261	0.045968	0.058832
Обратный	0.034077	0.052817	0.055640
Частичный	0.031072	2.738694	0.188024
Случайный	0.087489	1.685012	0.050851

Вывод

В первой части исследования график демонстрирует, что с ростом размера массива время выполнения сортировки увеличивается чрезвычайно быстро, следуя приблизительно кубической зависимости. Это указывает на низкую эффективность данного алгоритма при обработке больших наборов данных, поскольку его производительность резко ухудшается. Хотя на малых массивах могут наблюдаться некоторые расхождения между теоретическими прогнозами и практическими замерами из-за особенностей реализации, общая тенденция соответствует ожиданиям. Таким образом, для работы с крупными данными данный метод неприменим, и целесообразнее использовать более оптимизированные алгоритмы.

В сравнении с теоретической оценкой времени по сложности $O(N^3)$ экспериментальные данные показывают примерно такой же быстрый рост времени с увеличением размера массива, что подтверждает высокую вычислительную сложность алгоритма.

При сравнении сортировки Шелла и быстрой сортировки (quicksort) на различных типах массивов (случайном, отсортированном по возрастанию, по убыванию и специальном) разница во времени их работы оказалась незначительной. Это свидетельствует о том, что оба алгоритма демонстрируют высокую эффективность на небольших объемах данных: как быстрая сортировка, так и сортировка Шелла выполняются за доли миллисекунды, практически независимо от исходного порядка элементов.